STAR-D — ESP32-S3 Pin Assignments

Traced from VFINAL_PCB_Rexus/Main_PCB_STAR_D.kicad_sch and its five sub-sheets · cross-checked against SED v3-1 · REXUS 38 · 3 Nov 2026

What changed since the previous revision

The MCP23S17 I/O expander is **gone**. The REXUS flags, all three heater enables and the stepper enable are back on direct GPIOs, so `rs422_interface.c` and `gpio_driver.c` work as written. The Pi now has its own dedicated SPI bus, and an ISL83491 RS-422 transceiver has appeared. Three defects remain — see page 3.

Every ESP32-S3 pin that carries a signal

GPIO	CHIP	NET	CONNECTS TO	SOFTWARE OWNER	
GPIO1	6	LO_IN	J2.5 — REXUS lift-off flag, TP15	rs422_interface.c	D.35
GPIO2	7	SOE_IN	J2.6 — REXUS SOE flag, TP16	rs422_interface.c	D.35
GPIO3	8	I2C_SCL	J4.5 → SLF3S flow sensor, pull-up R_I2C_SCL2	i2c_interface.c	strap
GPIO4	9	GPIO_ADC	potentiometer R_ADC_POT1 + C17 (SED fig. 38)	— not written	?
GPIO5	10	STEP_M1	IC6.16 STEP — TMC2209, syringe pump	gpio_driver.c	ok
GPIO6	11	DIR_M1	IC6.19 DIR — TMC2209	gpio_driver.c	ok
GPIO7	12	I2C_SDA	J4.2 → SLF3S flow sensor, pull-up R_I2C_SCL1	i2c_interface.c	ok
GPIO8	13	EN_STEPPERS	IC6.2 ENN — TMC2209, **active LOW**	gpio_driver.c	ok
GPIO9	14	CS_ABP	J1.5 → ABP pressure sensor, off-board	(driver not written)	ok
GPIO10	15	CS_BME280	U2.2 — BME280, on-board (internal)	bme_driver.c	ok
GPIO11	16	CS_EXTERNAL_BME280	J7.6 → BME280, off-board (external)	bme_driver.c	ok
GPIO12	17	CS_LSM6DSM	U4.12 — IMU	imu_driver.c	ok
GPIO13	18	SPI_SCK	sensor bus — U2, U4, IC1, J1.3, J7.3, TP4	spi_interface.c	ok
GPIO14	19	SODS_IN	J2.7 — REXUS SODS flag, TP5	rs422_interface.c	D.35
GPIO17	23	SPI_MOSI	sensor bus — U2, U4, IC1, J7.4, TP6	spi_interface.c	ok
GPIO18	24	INT_LSM6DSM	U4.4 INT1 — IMU data-ready / event	imu_driver.c	new
GPIO19	25	USB-D−	R16 → J10 USB-C	(USB Serial/JTAG)	fixed
GPIO20	26	USB-D+	R17 → J10 USB-C	(USB Serial/JTAG)	fixed
GPIO21	27	CS_MAX31865	IC1.13 — PT100 front-end	pt100_driver.c	ok
GPIO33	38	HEATER1_EN	R_G1 100 R → Q7 gate (pull-down R_PD1)	gpio_driver.c	ok
GPIO34	39	CS_RASP	J3.1 — SPI **slave** select, driven by the Pi	spi_pi_link.c	ok
GPIO35	40	RASP_CLK	J3.2 — SPI clock in, from the Pi	spi_pi_link.c	ok
GPIO36	41	RASP_MOSI	J3.3 — SPI data in, from the Pi	spi_pi_link.c	ok
GPIO37	42	HEATER2_EN	R_G2 100 R → Q8 gate (pull-down R_PD2)	gpio_driver.c	ok
GPIO38	43	HEATER3_EN	**two** MOSFETs: R_G3→Q9 and R_G4→Q10	gpio_driver.c	?
GPIO39	44	STEP_IN4	IC5.4 (4B) — ULN2003, microscope stepper	gpio_driver.c	JTAG
GPIO40	45	STEP_IN3	IC5.3 (3B) — ULN2003	gpio_driver.c	JTAG
GPIO41	47	STEP_IN2	IC5.2 (2B) — ULN2003	gpio_driver.c	JTAG
GPIO42	48	STEP_IN1	IC5.1 (1B) — ULN2003	gpio_driver.c	JTAG
GPIO43	49	ESP_32_RO	IC3.2 RO — transceiver **receiver output**	uart_tcu.c	swap
GPIO44	50	ESP_32_DI	IC3.5 DI — transceiver **driver input**	uart_tcu.c	swap
GPIO45	51	SPI_MISO	sensor bus — U2.5, U4.1, IC1.14, J1.4, J7.5	spi_interface.c	strap
GPIO46	52	RASP_MISO	J3.4 — SPI data out, to the Pi	spi_pi_link.c	strap

GPIO numbers for chip pins 44–50 are not printed in the symbol, which names them MTCK/MTDO/MTDI/MTMS/U0TXD/U0RXD. They follow from the package order: pin 43 is GPIO38 and pin 51 is GPIO45, which leaves exactly GPIO39–44 for the six pins between them, in order. Chip pins 30–35 carry the in-package flash (IC2, W25Q01JV) and are untouched by any signal net. GPIO0 is unused.

Buses, connectors and what hangs off them

```
ESP32-S3 (U1)
  |
  +== SENSOR SPI  (ESP is MASTER)   SCK GPIO13 / MOSI GPIO17 / MISO GPIO45
  |       |-- CS GPIO10  U2   BME280        on-board
  |       |-- CS GPIO11  J7.6 BME280        off-board   (J7: 3=SCK 4=MOSI 5=MISO 6=CS)
  |       |-- CS GPIO12  U4   LSM6DSM  IMU  + INT1 -> GPIO18
  |       |-- CS GPIO21  IC1  MAX31865 PT100
  |       +-- CS GPIO9   J1.5 ABP pressure  off-board   (J1: 3=SCK 4=MISO 5=CS, no MOSI)
  |
  +== RASPBERRY PI SPI  (ESP is SLAVE -- SED table 4.13)      J3, 4 pins
  |       |-- CS_RASP   GPIO34   <- Pi drives
  |       |-- RASP_CLK  GPIO35   <- Pi drives
  |       |-- RASP_MOSI GPIO36   <- Pi drives
  |       +-- RASP_MISO GPIO46   -> ESP drives
  |           ( no handshake line, and no ground pin, on this connector )
  |
  +== I2C   SCL GPIO3 / SDA GPIO7 -> J4   SLF3S flow sensor   (J4: 2=SDA 5=SCL)
  |
  +== UART  GPIO43/44 -> IC3 ISL83491 -> J2.1-4  RS-422 to RXSM
  |           ~RE tied GND, DE tied 3V3  =>  always receiving, always driving
  |           R10 120 R across EXP_IN+/- = receive-side termination
  |
  +-- REXUS flags, direct, no isolation:  LO GPIO1 / SOE GPIO2 / SODS GPIO14  <- J2.5/6/7
  |
  +-- Heaters: GPIO33->Q7, GPIO37->Q8, GPIO38->Q9 AND Q10   (100 R gate, pull-down each)
  |
  +-- Steppers: TMC2209 IC6  STEP GPIO5 / DIR GPIO6 / ENN GPIO8  -> J5  (bipolar)
                ULN2003 IC5  IN1..4 GPIO42/41/40/39            -> J9  (unipolar, 4-phase)
```

Connector pinouts, as drawn

CONN.	PURPOSE	PINS
J2	REXUS service interface (9-way)	1 EXP_OUT− · 2 EXP_OUT+ · 3 EXP_IN+ · 4 EXP_IN− · 5 LO · 6 SOE · 7 SODS · 8–9 GND
J3	Raspberry Pi SPI (4-way)	1 CS · 2 CLK · 3 MOSI · 4 MISO
J1	ABP pressure sensor (5-way)	1–2 power · 3 SCK · 4 MISO · 5 CS
J4	SLF3S flow sensor (6-way)	2 SDA · 3–4 power · 5 SCL
J7	External BME280 (6-way)	1–2 power · 3 SCK · 4 MOSI · 5 MISO · 6 CS
J5	Bipolar stepper, from TMC2209	1 OB2 · 2 OB1 · 3 OA2 · 4 OA1
J9	Unipolar stepper, from ULN2003	1–4 coils A–D · 5 common
J10	USB-C	A6/B6 D+ · A7/B7 D−

Two steppers now, not one

IC6 (TMC2209) drives the syringe pump on STEP/DIR/ENN — three pins, the driver does the commutation. IC5 (ULN2003) drives the microscope stage directly: the software has to generate the four-phase sequence itself on GPIO42/41/40/39, in order, with its own timing. These are different jobs and `gpio_driver.c` currently handles neither. The ULN2003 has no enable pin, so its coils are energised whenever the software leaves an input high — which matters for both heat and current budget.

Nets traced by computing each symbol's absolute pin coordinates, joining them through wire segments and junctions, and naming the result from the labels landing on it. 6 sheets, 381 wires, 540 pins, 235 nets. Sub-sheets are bonded by global labels, not sheet pins.

Cross-check against SED v3-1

1 Requirement D.35 is not met by this board

D.35 — *“Optocoupler shall be used on control interface lines”*. Verified by **DV-I03**: *“verify optocoupler isolation on control lines (functional check, not presence only)”*. SED Figure 37 is still captioned *“Optocoupler interface between REXUS service signals and onboard control electronics”*.

The previous revision had three VO615A-4 optocouplers with pull-ups. **This revision has none.** `LO_IN` runs J2.5 → GPIO1 with only a test point in between; the same for SOE and SODS. DV-I03 cannot pass as the board stands.

There is a second problem underneath it. The REXUS ICD specifies the flags as low-side switches and states *“the recommended current for each flag is between 2 and 50 mA”*. With no pull-up resistor on the board, the only pull-up available is the ESP's internal one at roughly 45 kΩ, giving about **73 µA** — far below the ICD minimum. The flag inputs are non-compliant and electrically marginal at the same time.

2 The RS-422 data pins appear swapped

`ESP_32_R0` carries IC3 pin 2 (RO, the transceiver's receiver *output*) to U0TXD, and `ESP_32_DI` carries IC3 pin 5 (DI, the driver *input*) to U0RXD. RO is an output and U0TXD is an output: they contend. The ISL83491 symbol follows the standard full-duplex pinout, so this is not a symbol error.

Recoverable in software — the S3's GPIO matrix can put UART RX on GPIO43 and TX on GPIO44. But the ROM bootloader drives GPIO43 as U0TXD at every reset, into IC3's RO output, before any of our code runs. The clean fix is to swap the two nets on the board. If that is not possible, the console must be moved to USB Serial/JTAG on J10 and the contention accepted for the duration of ROM boot.

3 SED table 4.13 and `protocol.h` disagree on one row

SED TABLE 4.13	PROTOCOL.H	STATUS
Clock 1 MHz	`SPI_CLOCK_HZ 1000000`	agrees
Mode 0 (CPOL 0, CPHA 0)	`SPI_MODE 0`	agrees
Master/Slave = Pi/ESP	`ESP configured as slave`	agrees — settles the open question
MSB first, 8-bit words	`defaults`	agrees
Single slave, CS active low	`CS_RASP on GPIO34`	agrees
Sensor transfer 10 Hz	`SPI_TRANSACTION_PERIOD_MS 100`	agrees
Status exchange 1 Hz	`telemetry self-limits to 1 Hz`	agrees
Sequence order + CRC per frame	`implemented`	agrees
Degraded after 3 invalid, cleared after 5 valid	**`MISS_COUNT_DEGRADED 30` `SUCCESS_COUNT… 50`**	**contradicts**

The code was deliberately changed: 3 misses at 10 Hz is 300 ms, which a Linux machine flushing video will exceed while perfectly healthy, so the SED's figures would raise a false degraded flag in normal operation. The reasoning holds, but the SED has not been updated. Either amend table 4.13 to 30/50 with that justification, or revert the code. Leaving the two different invites exactly the finding a review panel looks for.

Where the board and the SED agree

- TMC2209 replaces the A4988 (CDR action on driver noise), controlled by STEP/DIR/EN exactly as SED table 4.5 states. The sub-sheet is still *named* `A4988.kicad_sch`, but the part placed is a TMC2209-LA.
- ULN2003ADR present and described as a “GPIO-based 4-phase stepper motor” driver.
- ISL83491 RS-422 transceiver present, matching SED figure 40 and the power budget line item.
- Both BME280s, the PT100 via MAX31865, the LSM6DSM and the ABP are on SPI; the SLF3S is the only I²C device — exactly as SED §4.8.2.2 says.
- Every heater gate has its own 100 Ω series resistor and its own pull-down, so the heaters are off while the ESP's pins are high-impedance at reset. That is the correct failure direction and is worth stating in the SED safety section.
- `GPIO_ADC` is the potentiometer of SED figure 38 — not a bus voltage monitor. The `voltage_mV` field in `sensors.c` has no source on this board.

What this means for the software

Blocked until hardware answers

- **The Pi link has no handshake line.** J3 has four pins: CS, CLK, MOSI, MISO. Because the ESP is the slave, its transmit buffer must be armed *before* the Pi starts clocking, and `spi_pi_link.c` signals that with a handshake GPIO. There is no wire for it. Either add a fifth pin, or accept that every frame the Pi reads is one cycle (100 ms) stale and remove the handshake from the design. This needs deciding before the link can be tested.
- **J3 has no ground pin either.** A 1 MHz SPI link between two boards with no dedicated return in the connector will work on the bench and may not work on the stand. Confirm the return path through the power harness, or add the pin.
- **The RO/DI swap** decides whether `uart_tcu.c` can use the default UART pins or must remap through the GPIO matrix, and whether the IDF console has to move to USB.
- **The optocouplers** decide whether `rs422_interface.c` keeps its active-LOW polarity. Re-adding isolation may invert the sense, and polarity is the one thing in that file that cannot be guessed.

Ready to write once the pins above are fixed in the header

- `gpio_driver.c` — three heater enables, TMC2209 STEP/DIR/ENN, and a new four-phase sequencer for the ULN2003. Note `HEATER3_EN` drives **two** MOSFETs (Q9 and Q10), so the software's "three channels" are four physical heaters; confirm this is intended before the thermal budget is redone.
- `imu_driver.c` — INT1 is now wired to GPIO18, so the IMU can be interrupt-driven instead of polled. Worth doing: it is the only sensor whose timing matters for launch detection.
- `spi_interface.c` — one master bus, six chip selects, all on ordinary GPIOs.

Strapping pins and JTAG — confirm before the first board powers on

- **GPIO45 carries SPI_MISO** and is the VDD_SPI strap: LOW at reset selects 3.3 V for the flash, HIGH selects 1.8 V. With every chip select idle at boot, MISO floats, and a floating strap pin is how an intermittent brick happens. Add an explicit pull-down on `SPI_MIS0`.
- **GPIO46 carries RASP_MISO** and is the ROM-message strap. It defaults low, which is the harmless state, but confirm the Pi end does not hold it high at reset.
- **GPIO3 carries I2C_SCL** and is the JTAG-source strap. The I²C pull-up holds it high through reset. Confirm that selection is the intended one.
- **GPIO39–42 are the JTAG pins**, now used for the ULN2003. Hardware JTAG debugging is gone. USB Serial/JTAG on J10 remains, so this is acceptable — but it means the USB-C connector is now the only debug route and must stay accessible during integration.

Sources: VFINAL_PCB_Rexus schematic set (3 Nov 2026) for every pin and net; RX38_STAR-D_SED v3-1 (14 Sep 2026) tables 4.5 and 4.13, requirement D.35, verification DV-I03 and figures 37–43; DLR REXUS ICD v1.1 table 2.2 and the flag electrical properties. Where the schematic and the SED disagree, neither is assumed correct — both are listed.